

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине: «Интеллектуальный анализ данных»

Тема: «Предобучение нейронных сетей с использованием RBM»

Выполнила:

Студент 4 курса

Группы ИИ-26

Карпович К.А.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: научиться осуществлять предобучение нейронных сетей с помощью RBM

Общее задание

1. Взять за основу нейронную сеть из лабораторной работы №3. Выполнить обучение с предобучением, используя стек ограниченных машин Больцмана (RBM – Restricted Boltzmann Machine), алгоритм которого изложен в лекции. Условие останова (например, по количеству эпох) при обучении отдельных слоев как RBM выбрать самостоятельно.
2. Сравнить результаты, полученные при
 - обучении без предобучения (ЛР 3);
 - обучении с предобучением, используя автоэнкодерный подход (ЛР3);
 - обучении с предобучением, используя RBM.
3. Обучить модели на данных из ЛР 2, сравнить результаты по схеме из пункта 2;
4. Сделать выводы, оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№	Выборка	Тип задачи	Целевая переменная
6	https://archive.ics.uci.edu/dataset/265/physicochemical+properties+of+protein+tertiary+structure	регрессия	RMSD

Код программы:

```
import copy
import os
import time

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
from scipy import stats
from sklearn.datasets import load_breast_cancer
from sklearn.metrics import (accuracy_score, confusion_matrix, f1_score,
                             mean_absolute_error, mean_squared_error,
                             precision_score, r2_score, recall_score,
                             ConfusionMatrixDisplay)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

CASP_PATH = "CASP.csv"
OUT_DIR = "results_lr4"
SHOW_PLOTS = True
SEEDS = [0, 1, 2, 3, 4]
BATCH = 256

AE_EPOCHS = 30
AE_LR = 1e-3

RBM_EPOCHS = 30
RBM_LR_GAUSS = 5e-3
RBM_LR_BINARY = 5e-2
RBM_CD_K = 1
RBM_WEIGHT_DECAY = 1e-4
RBM_MOMENTUM_START = 0.5
RBM_MOMENTUM_END = 0.9
RBM_MOMENTUM_SWITCH = 5

HEAD_EPOCHS = 30
FT_LR = 1e-3
MAPE_MIN_Y = 1.0

DATASETS = {
```

```
"CASP (RMSD, регрессия)": dict(task="regression", hidden=[64, 32, 16, 8], batch=256, epochs=60, fracs=[1.0, 0.05]),
"WDBC (рак груди, классификация)": dict(task="classification", hidden=[64, 32, 16, 8], batch=32, epochs=100, fracs=[1.0, 0.25]),
}
```

```
METHOD_BASE = "Без предобучения"
METHOD_AE_ONLY = "Автоэнкодеры: только предобучение"
METHOD_AE = "Автоэнкодеры + дообучение"
METHOD_RBM_ONLY = "RBM: только предобучение"
METHOD_RBM = "RBM + дообучение"
```

```
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
def load_casp():
    if os.path.exists(CASP_PATH):
        df = pd.read_csv(CASP_PATH)
    else:
        try:
            from ucimlrepo import fetch_ucirepo
            ds = fetch_ucirepo(id=265)
            df = pd.concat([ds.data.targets, ds.data.features], axis=1)
        except Exception as e:
            raise FileNotFoundError(
                f"Не найден {CASP_PATH}. Скачайте датасет с "
                "https://archive.ics.uci.edu/dataset/265 и положите CASP.csv рядом со скриптом."
            ) from e
    X = df.drop(columns=["RMSD"]).values.astype(np.float32)
    y = df["RMSD"].values.astype(np.float32)
    return X, y, None
```

```
def load_wdbc():
    data = load_breast_cancer()
    return data.data.astype(np.float32), data.target.astype(np.float32), list(data.target_names)
```

```
class MLP(nn.Module):
    def __init__(self, dims):
        super().__init__()
        self.layers = nn.ModuleList(nn.Linear(a, b) for a, b in zip(dims[:-1], dims[1:]))
```

```
    def features(self, x):
        for layer in self.layers[:-1]:
            x = torch.relu(layer(x))
        return x
```

```
    def forward(self, x):
        return self.layers[-1](self.features(x))
```

```
class RBM:
    def __init__(self, n_visible, n_hidden, gaussian_visible):
        self.gaussian_visible = gaussian_visible
        self.W = torch.randn(n_visible, n_hidden, device=DEVICE) * 0.01
        self.b = torch.zeros(n_visible, device=DEVICE)
        self.c = torch.zeros(n_hidden, device=DEVICE)
        self.vW = torch.zeros_like(self.W)
        self.vb = torch.zeros_like(self.b)
        self.vc = torch.zeros_like(self.c)
```

```
    def hidden_prob(self, v):
        return torch.sigmoid(v @ self.W + self.c)
```

```
    def visible_mean(self, h):
        z = h @ self.W.t() + self.b
        return z if self.gaussian_visible else torch.sigmoid(z)
```

```
    def contrastive_divergence(self, v0, k, lr, momentum, weight_decay):
        n = v0.shape[0]
        h0_prob = self.hidden_prob(v0)
        h = torch.bernoulli(h0_prob)
        for step in range(k):
            vk = self.visible_mean(h)
            hk_prob = self.hidden_prob(vk)
            if step < k - 1:
                h = torch.bernoulli(hk_prob)
            dW = (v0.t() @ h0_prob - vk.t() @ hk_prob) / n - weight_decay * self.W
            db = (v0 - vk).mean(dim=0)
```

```

        dc = (h0_prob - hk_prob).mean(dim=0)
        self.vW = momentum * self.vW + lr * dW
        self.vb = momentum * self.vb + lr * db
        self.vc = momentum * self.vc + lr * dc
        self.W += self.vW
        self.b += self.vb
        self.c += self.vc
        return F.mse_loss(vk, v0).item()

def batches(n, bs, gen):
    perm = torch.randperm(n, generator=gen)
    for i in range(0, n, bs):
        yield perm[i:i + bs].to(DEVICE)

def sup_loss(task, out, target):
    out = out.squeeze(1)
    if task == "regression":
        return F.mse_loss(out, target)
    return F.binary_cross_entropy_with_logits(out, target)

def pretrain_with_autoencoders(model, X, gen):
    ae_history, cur = [], X
    for k, layer in enumerate(model.layers[:-1]):
        decoder = nn.Linear(layer.out_features, layer.in_features).to(DEVICE)
        opt = torch.optim.Adam(list(layer.parameters()) + list(decoder.parameters()), lr=AE_LR)
        losses = []
        for _ in range(AE_EPOCHS):
            total = 0.0
            for idx in batches(len(cur), BATCH, gen):
                xb = cur[idx]
                rec = decoder(torch.relu(layer(xb)))
                loss = F.mse_loss(rec, xb)
                opt.zero_grad()
                loss.backward()
                opt.step()
                total += loss.item() * len(idx)
            losses.append(total / len(cur))
        ae_history.append(losses)
        with torch.no_grad():
            cur = torch.relu(layer(cur))
    return ae_history

def pretrain_with_rbm(model, X, gen):
    rbm_history, cur = [], X
    for k, layer in enumerate(model.layers[:-1]):
        gaussian = (k == 0)
        lr = RBM_LR_GAUSS if gaussian else RBM_LR_BINARY
        rbm = RBM(layer.in_features, layer.out_features, gaussian)
        losses = []
        for epoch in range(RBM_EPOCHS):
            momentum = RBM_MOMENTUM_START if epoch < RBM_MOMENTUM_SWITCH else RBM_MOMENTUM_END
            total = 0.0
            for idx in batches(len(cur), BATCH, gen):
                err = rbm.contrastive_divergence(cur[idx], RBM_CD_K, lr, momentum, RBM_WEIGHT_DECAY)
                total += err * len(idx)
            losses.append(total / len(cur))
        rbm_history.append(losses)
        with torch.no_grad():
            layer.weight.copy_(rbm.W.t())
            layer.bias.copy_(rbm.c)
            cur = rbm.hidden_prob(cur)
    return rbm_history

def train_last_layer(model, X, y, task, gen):
    with torch.no_grad():
        feats = model.features(X)
    head = model.layers[-1]
    opt = torch.optim.Adam(head.parameters(), lr=FT_LR)
    for _ in range(HEAD_EPOCHS):
        for idx in batches(len(feats), BATCH, gen):
            loss = sup_loss(task, head(feats[idx]), y[idx])
            opt.zero_grad()
            loss.backward()

```

```

    opt.step()

@torch.no_grad()
def predict_raw(model, X):
    model.eval()
    out = model(X).squeeze(1).cpu().numpy()
    model.train()
    return out

def train_supervised(model, Xtr, ytr, Xte, yte, task, epochs, gen):
    opt = torch.optim.Adam(model.parameters(), lr=FT_LR)
    hist = {"train": [], "test": []}
    for _ in range(epochs):
        total = 0.0
        for idx in batches(len(Xtr), BATCH, gen):
            loss = sup_loss(task, model(Xtr[idx]), ytr[idx])
            opt.zero_grad()
            loss.backward()
            opt.step()
            total += loss.item() * len(idx)
        hist["train"].append(total / len(Xtr))
        with torch.no_grad():
            hist["test"].append(sup_loss(task, model(Xte), yte).item())
    return hist

def regression_metrics(y_true, y_pred):
    mask = y_true >= MAPE_MIN_Y
    mape = np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100
    return {"MAE": mean_absolute_error(y_true, y_pred),
            "RMSE": np.sqrt(mean_squared_error(y_true, y_pred)),
            "R2": r2_score(y_true, y_pred),
            "MAPE, %": mape}

def classification_metrics(y_true, prob):
    pred = (prob >= 0.5).astype(int)
    return {"Accuracy": accuracy_score(y_true, pred),
            "Precision": precision_score(y_true, pred, zero_division=0),
            "Recall": recall_score(y_true, pred, zero_division=0),
            "F1": f1_score(y_true, pred, zero_division=0)}

def evaluate(model, X, y_true, task, y_scaler):
    raw = predict_raw(model, X)
    if task == "regression":
        pred = y_scaler.inverse_transform(raw.reshape(-1, 1)).ravel()
        return regression_metrics(y_true, pred), pred
    prob = 1 / (1 + np.exp(-np.clip(raw, -50, 50)))
    return classification_metrics(y_true, prob), prob

def run_seed(X_all, y_all, task, hidden, epochs, frac, seed):
    strat = y_all if task == "classification" else None
    Xtr, Xte, ytr, yte = train_test_split(X_all, y_all, test_size=0.2, random_state=seed, stratify=strat)
    if frac < 1.0:
        keep = np.random.RandomState(seed).permutation(len(Xtr))[:max(int(len(Xtr) * frac), 20)]
        Xtr, ytr = Xtr[keep], ytr[keep]
    x_scaler = StandardScaler().fit(Xtr)
    Xtr_s, Xte_s = x_scaler.transform(Xtr), x_scaler.transform(Xte)
    y_scaler = None
    ytr_s, yte_s = ytr, yte
    if task == "regression":
        y_scaler = StandardScaler().fit(ytr.reshape(-1, 1))
        ytr_s = y_scaler.transform(ytr.reshape(-1, 1)).ravel()
        yte_s = y_scaler.transform(yte.reshape(-1, 1)).ravel()
    t = lambda a: torch.tensor(np.asarray(a), dtype=torch.float32, device=DEVICE)
    Xtr_t, Xte_t, ytr_t, yte_t = t(Xtr_s), t(Xte_s), t(ytr_s), t(yte_s)

    dims = [X_all.shape[1]] + hidden + [1]
    torch.manual_seed(seed)
    base = MLP(dims).to(DEVICE)
    ae_net = copy.deepcopy(base)
    rbm_net = copy.deepcopy(base)

    hist_base = train_supervised(base, Xtr_t, ytr_t, Xte_t, yte_t, task, epochs, torch.Generator().manual_seed(seed))

```

```

m_base, out_base = evaluate(base, Xte_t, yte, task, y_scaler)

gen = torch.Generator().manual_seed(seed)
ae_hist = pretrain_with_autoencoders(ae_net, Xtr_t, gen)
train_last_layer(ae_net, Xtr_t, ytr_t, task, gen)
m_ae_only, _ = evaluate(ae_net, Xte_t, yte, task, y_scaler)
hist_ae = train_supervised(ae_net, Xtr_t, ytr_t, Xte_t, yte_t, task, epochs, gen)
m_ae, out_ae = evaluate(ae_net, Xte_t, yte, task, y_scaler)

torch.manual_seed(seed + 1000)
gen = torch.Generator().manual_seed(seed)
rbm_hist = pretrain_with_rbm(rbm_net, Xtr_t, gen)
train_last_layer(rbm_net, Xtr_t, ytr_t, task, gen)
m_rbm_only, _ = evaluate(rbm_net, Xte_t, yte, task, y_scaler)
hist_rbm = train_supervised(rbm_net, Xtr_t, ytr_t, Xte_t, yte_t, task, epochs, gen)
m_rbm, out_rbm = evaluate(rbm_net, Xte_t, yte, task, y_scaler)

return dict(metrics={METHOD_BASE: m_base,
                    METHOD_AE_ONLY: m_ae_only,
                    METHOD_AE: m_ae,
                    METHOD_RBM_ONLY: m_rbm_only,
                    METHOD_RBM: m_rbm},
            hist_base=hist_base, hist_ae=hist_ae, hist_rbm=hist_rbm,
            ae_hist=ae_hist, rbm_hist=rbm_hist,
            y_true=yte, out_base=out_base, out_ae=out_ae, out_rbm=out_rbm)

def make_figure(name, frac, task, runs, class_names, path):
    first = runs[0]
    plt.rcParams.update({"font.size": 10})
    fig, axes = plt.subplots(2, 3, figsize=(18, 10), layout="constrained")
    ax = axes.ravel()

    for k, losses in enumerate(first["ae_hist"]):
        ax[0].plot(losses, label=f"Слой {k + 1}")
    ax[0].set_title("Автоэнкодеры: потери реконструкции (MSE)")
    ax[0].set_xlabel("Эпоха")
    ax[0].set_ylabel("MSE")
    ax[0].set_yscale("log")
    ax[0].legend()
    ax[0].grid(alpha=0.3)

    for k, losses in enumerate(first["rbm_hist"]):
        ax[1].plot(losses, label=f"RBM {k + 1}")
    ax[1].set_title("RBM: ошибка реконструкции (MSE, CD-1)")
    ax[1].set_xlabel("Эпоха")
    ax[1].set_ylabel("MSE")
    ax[1].set_yscale("log")
    ax[1].legend()
    ax[1].grid(alpha=0.3)

    for key, label in (("hist_base", "Без предобучения"),
                      ("hist_ae", "Автоэнкодеры"),
                      ("hist_rbm", "RBM")):
        ax[2].plot(np.mean([r[key]["test"] for r in runs], axis=0), label=label)
    ax[2].set_title("Ошибка на тесте (среднее по запускам)")
    ax[2].set_xlabel("Эпоха")
    ax[2].set_ylabel("MSE (норм.)" if task == "regression" else "BCE")
    ax[2].legend()
    ax[2].grid(alpha=0.3)

    panels = ((ax[3], "out_base", "Без предобучения"),
              (ax[4], "out_ae", "Автоэнкодеры"),
              (ax[5], "out_rbm", "RBM"))
    for a, key, title in panels:
        if task == "regression":
            n = min(3000, len(first["y_true"]))
            a.scatter(first["y_true"][:n], first[key][:n], s=5, alpha=0.4)
            lim = [0, max(first["y_true"].max(), first[key].max())]
            a.plot(lim, lim, "r--")
            a.set_xlabel("Истинный RMSD")
            a.set_ylabel("Предсказанный RMSD")
            a.set_title(f"{title}: прогноз и факт")
        else:
            cm = confusion_matrix(first["y_true"], (first[key] >= 0.5).astype(int))
            ConfusionMatrixDisplay(cm, display_labels=class_names).plot(ax=a, colorbar=False)
            a.set_xlabel("Предсказанный класс")
            a.set_ylabel("Истинный класс")

```

```

a.set_title(f'{title}: матрица ошибок')

fig.suptitle(f'{name}\nдоля обучающей выборки = {frac}', fontsize=14)
fig.savefig(path, dpi=150)

def paired_tests(df, metric_cols, task):
    main = "RMSE" if task == "regression" else "F1"
    pairs = ((METHOD_AE, METHOD_BASE), (METHOD_RBM, METHOD_BASE), (METHOD_RBM, METHOD_AE))
    rows = []
    for (ds, frac), g in df.groupby(["Датасет", "Доля train"], sort=False):
        if ("RMSE" in g.columns and g["RMSE"].notna().any()) != (task == "regression"):
            continue
        piv = g.pivot(index="seed", columns="Метод", values=main)
        for a, b in pairs:
            diff = piv[a] - piv[b]
            if np.allclose(diff, 0):
                p = 1.0
            else:
                p = stats.ttest_rel(piv[a], piv[b]).pvalue
            rows.append({"Датасет": ds, "Доля train": frac, "Метрика": main,
                        "Сравнение": f'{a} vs {b}',
                        "Разность средних": diff.mean(), "p-value": p})
    return pd.DataFrame(rows)

def main():
    global BATCH
    os.makedirs(OUT_DIR, exist_ok=True)
    print(f'Устройство: {DEVICE}')
    loaders = {"CASP (RMSD, регрессия)": load_casp, "WDBC (рак груди, классификация)": load_wdbc}
    rows = []
    tasks = {}
    for name, cfg in DATASETS.items():
        X, y, class_names = loaders[name]()
        task = cfg["task"]
        tasks[name] = task
        BATCH = cfg["batch"]
        print(f'\n{int(* 70)\n{name}: {X.shape[0]} объектов, {X.shape[1]} признаков')
        print(f'Архитектура: {[X.shape[1]] + cfg["hidden"] + [1]}')
        for frac in cfg["fracs"]:
            runs = []
            t0 = time.time()
            for seed in SEEDS:
                r = run_seed(X, y, task, cfg["hidden"], cfg["epochs"], frac, seed)
                runs.append(r)
            for method, m in r["metrics"].items():
                rows.append({"Датасет": name, "Доля train": frac, "Метод": method, "seed": seed, **m})
            print(f' доля={frac}, seed={seed} готово ({time.time() - t0:.0f} c)')
            make_figure(name, frac, task, runs, class_names,
                        os.path.join(OUT_DIR, f'{name.split()[0]}_frac{frac}.png'))

df = pd.DataFrame(rows)
df.to_csv(os.path.join(OUT_DIR, "all_runs.csv"), index=False, encoding="utf-8-sig")
metric_cols = [c for c in df.columns if c not in ("Датасет", "Доля train", "Метод", "seed")]
summary = df.groupby(["Датасет", "Доля train", "Метод"], sort=False)[metric_cols].agg(["mean", "std"])
summary.to_csv(os.path.join(OUT_DIR, "summary.csv"), encoding="utf-8-sig")

pd.set_option("display.width", 250, "display.max_columns", 50, "display.max_colwidth", 80)
print("\n" + "=" * 70 + "\nИТОГОВЫЕ РЕЗУЛЬТАТЫ (mean по запускам; std в summary.csv)\n" + "=" * 70)
for (ds, frac), g in df.groupby(["Датасет", "Доля train"], sort=False):
    print(f'\n{ds}, доля обучающей выборки = {frac}')
    cols = [c for c in metric_cols if g[c].notna().any()]
    print(g.groupby("Метод", sort=False)[cols].mean().round(4).to_string())

tests = pd.concat([paired_tests(df, metric_cols, "regression"),
                  paired_tests(df, metric_cols, "classification")], ignore_index=True)
tests.to_csv(os.path.join(OUT_DIR, "paired_tests.csv"), index=False, encoding="utf-8-sig")
print("\n" + "=" * 70 + "\nПАРНЫЙ t-ТЕСТ ПО ЗАПУСКАМ (основная метрика)\n" + "=" * 70)
print(tests.round(4).to_string(index=False))

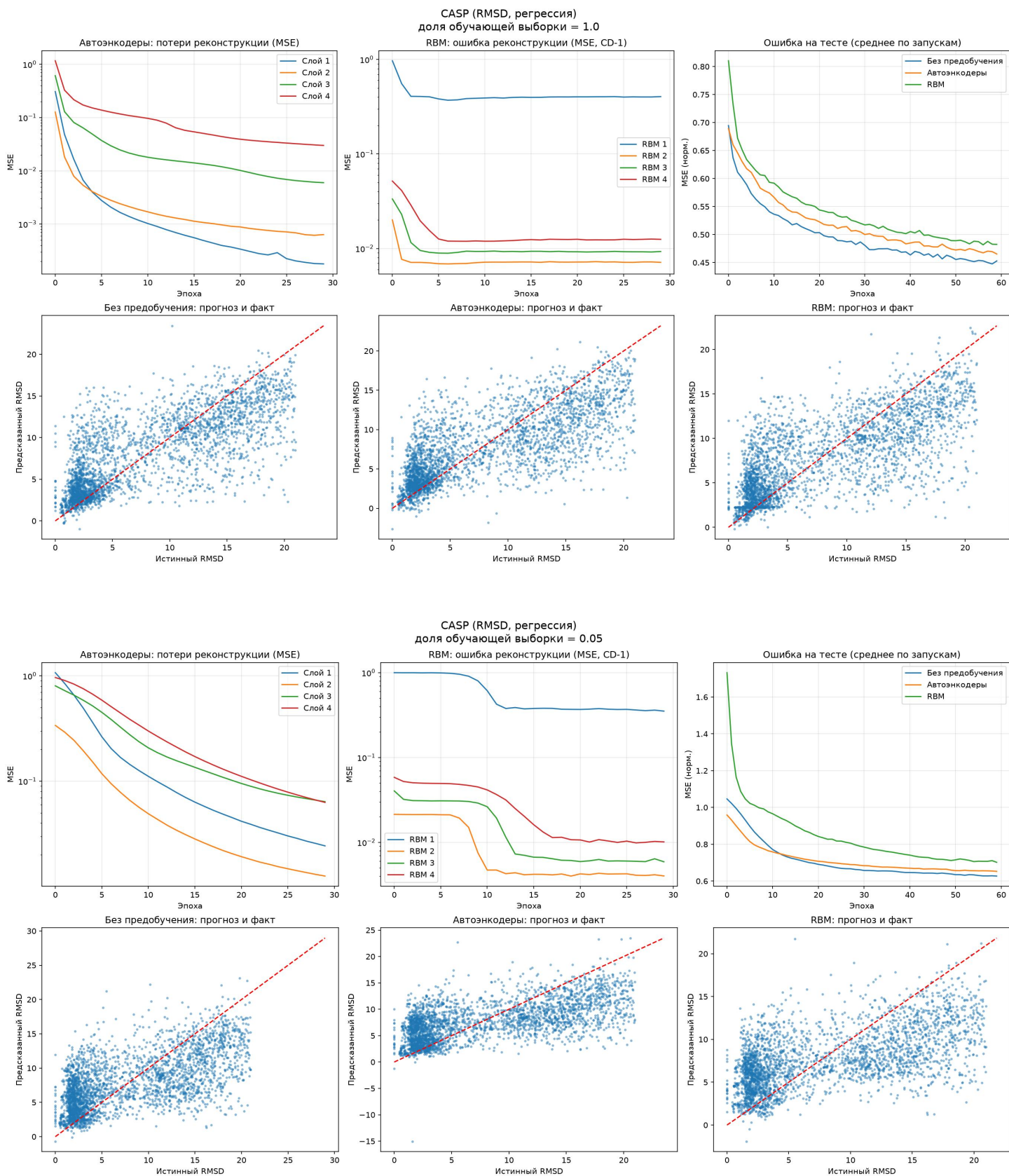
print(f'\nГрафики и таблицы сохранены в папку '{OUT_DIR}''')
if SHOW_PLOTS:
    plt.show()

if __name__ == "__main__":
    main()

```

Результат программы:

Устройство: cpu	ИТОГОВЫЕ РЕЗУЛЬТАТЫ (mean по запускам; std в summary.csv)				
=====					
CASP (RMSD, регрессия): 45730 объектов, 9 признаков					
CASP (RMSD, регрессия), доля обучающей выборки = 1.0					
Архитектура: [9, 64, 32, 16, 8, 1]		MAE	RMSE	R2	MAPE, %
доля=1.0, seed=0 готово (73 с)	Метод				
доля=1.0, seed=1 готово (129 с)	Без предобучения	2.9935	4.1177	0.5461	63.9108
доля=1.0, seed=2 готово (185 с)	Автоэнкодеры: только предобучение	4.6526	5.4656	0.2002	113.6060
доля=1.0, seed=3 готово (241 с)]	Автоэнкодеры + дообучение	3.1069	4.1743	0.5336	67.9738
доля=1.0, seed=4 готово (297 с)	RBM: только предобучение	5.1164	5.9462	0.0534	127.4437
доля=0.05, seed=0 готово (3 с)	RBM + дообучение	3.1900	4.2507	0.5164	72.9759
доля=0.05, seed=1 готово (7 с)	CASP (RMSD, регрессия), доля обучающей выборки = 0.05				
доля=0.05, seed=2 готово (10 с)		MAE	RMSE	R2	MAPE, %
доля=0.05, seed=3 готово (13 с)	Метод				
доля=0.05, seed=4 готово (16 с)	Без предобучения	3.8451	4.8410	0.3728	91.4554
=====					
WDVC (рак груди, классификация): 569 объектов, 30 признаков	Автоэнкодеры: только предобучение	5.4860	6.0915	0.0069	137.8876
Архитектура: [30, 64, 32, 16, 8, 1]	Автоэнкодеры + дообучение	3.9883	4.9397	0.3469	97.1570
доля=1.0, seed=0 готово (8 с)	RBM: только предобучение	7.0413	9.0457	-1.5856	169.8511
доля=1.0, seed=1 готово (15 с)	RBM + дообучение	4.2025	5.1214	0.2980	103.2004
доля=1.0, seed=2 готово (23 с)	WDVC (рак груди, классификация), доля обучающей выборки = 1.0				
доля=1.0, seed=3 готово (30 с)		Accuracy	Precision	Recall	F1
доля=1.0, seed=4 готово (38 с)	Метод				
доля=0.25, seed=0 готово (2 с)	Без предобучения	0.9754	0.9727	0.9889	0.9807
доля=0.25, seed=1 готово (4 с)	Автоэнкодеры: только предобучение	0.7982	0.7819	0.9694	0.8620
доля=0.25, seed=2 готово (6 с)	Автоэнкодеры + дообучение	0.9754	0.9727	0.9889	0.9807
доля=0.25, seed=3 готово (8 с)	RBM: только предобучение	0.8965	0.9042	0.9389	0.9207
доля=0.25, seed=4 готово (11 с)	RBM + дообучение	0.9737	0.9752	0.9833	0.9792
=====					
WDVC (рак груди, классификация), доля обучающей выборки = 0.25					
	Accuracy	Precision	Recall	F1	
Метод					
Без предобучения	0.9544	0.9645	0.9639	0.9638	
Автоэнкодеры: только предобучение	0.6035	0.5416	0.7417	0.6219	
Автоэнкодеры + дообучение	0.9561	0.9646	0.9667	0.9652	
RBM: только предобучение	0.5965	0.6225	0.7167	0.6572	
RBM + дообучение	0.8895	0.9037	0.9639	0.9253	
=====					
ПАРНЫЙ t-ТЕСТ ПО ЗАПУСКАМ (основная метрика)					
=====					
Датасет	Доля train	Метрика	Сравнение	Разность средних	p-value
CASP (RMSD, регрессия)	1.00	RMSE	Автоэнкодеры + дообучение vs Без предобучения	0.0566	0.0371
CASP (RMSD, регрессия)	1.00	RMSE	RBM + дообучение vs Без предобучения	0.1329	0.0006
CASP (RMSD, регрессия)	1.00	RMSE	RBM + дообучение vs Автоэнкодеры + дообучение	0.0763	0.0112
CASP (RMSD, регрессия)	0.05	RMSE	Автоэнкодеры + дообучение vs Без предобучения	0.0987	0.0826
CASP (RMSD, регрессия)	0.05	RMSE	RBM + дообучение vs Без предобучения	0.2804	0.0023
CASP (RMSD, регрессия)	0.05	RMSE	RBM + дообучение vs Автоэнкодеры + дообучение	0.1817	0.0002
WDVC (рак груди, классификация)	1.00	F1	Автоэнкодеры + дообучение vs Без предобучения	0.0000	1.0000
WDVC (рак груди, классификация)	1.00	F1	RBM + дообучение vs Без предобучения	-0.0015	0.6933
WDVC (рак груди, классификация)	1.00	F1	RBM + дообучение vs Автоэнкодеры + дообучение	-0.0015	0.6933
WDVC (рак груди, классификация)	0.25	F1	Автоэнкодеры + дообучение vs Без предобучения	0.0014	0.3739
WDVC (рак груди, классификация)	0.25	F1	RBM + дообучение vs Без предобучения	-0.0385	0.3307
WDVC (рак груди, классификация)	0.25	F1	RBM + дообучение vs Автоэнкодеры + дообучение	-0.0399	0.3111
Графики и таблицы сохранены в папку 'results_lr4'					



Сравнение результатов на выборке CASP (регрессия)

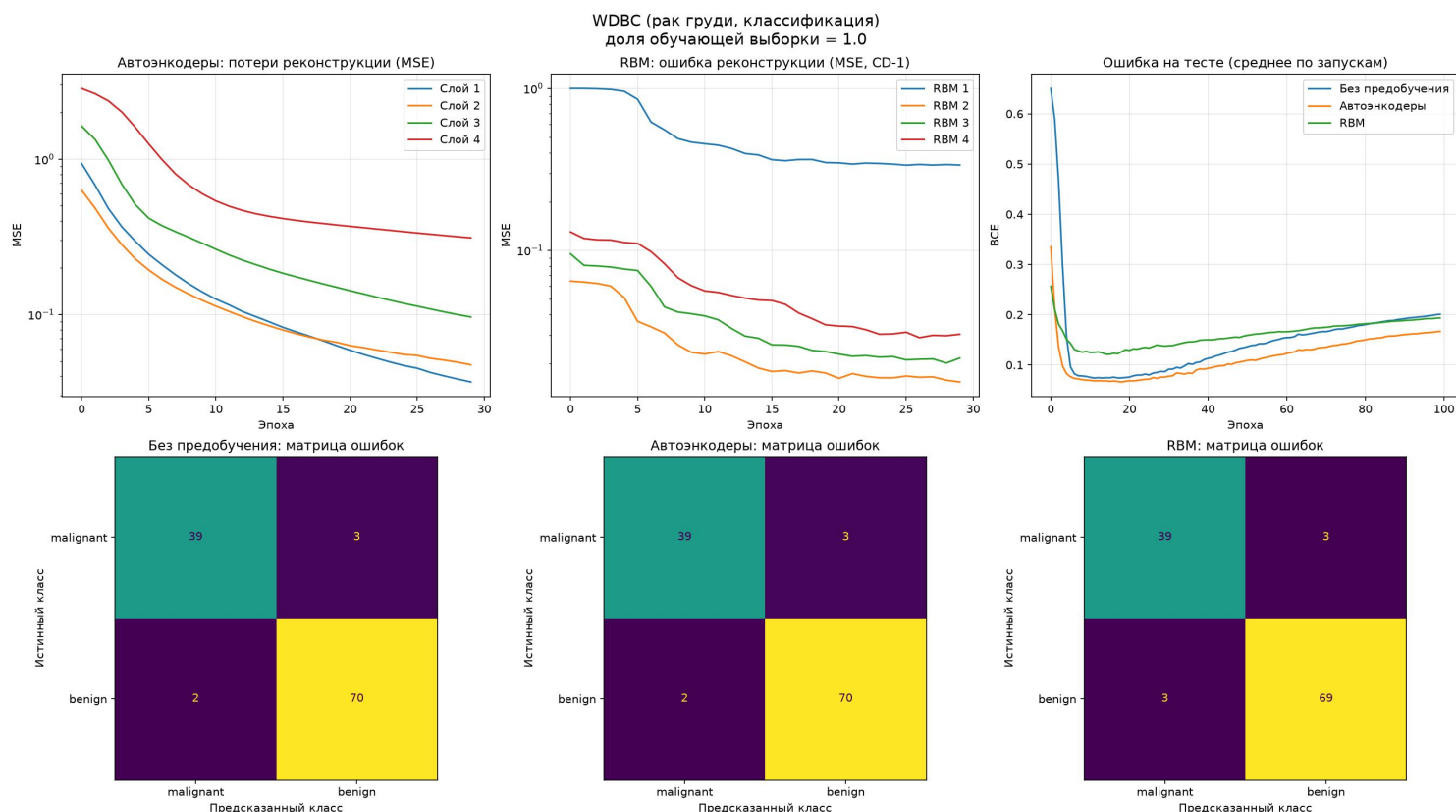
Сравнивались три подхода: обучение без предобучения (ЛР 3), предобучение автоэнкодерами (ЛР 3) и предобучение стеком RBM. Все три модели имели архитектуру 9-64-32-16-8-1 и стартовали с одинаковых начальных весов. Эксперимент повторён на 5 разбиениях (train/test = 80/20), в таблице приведены средние значения.

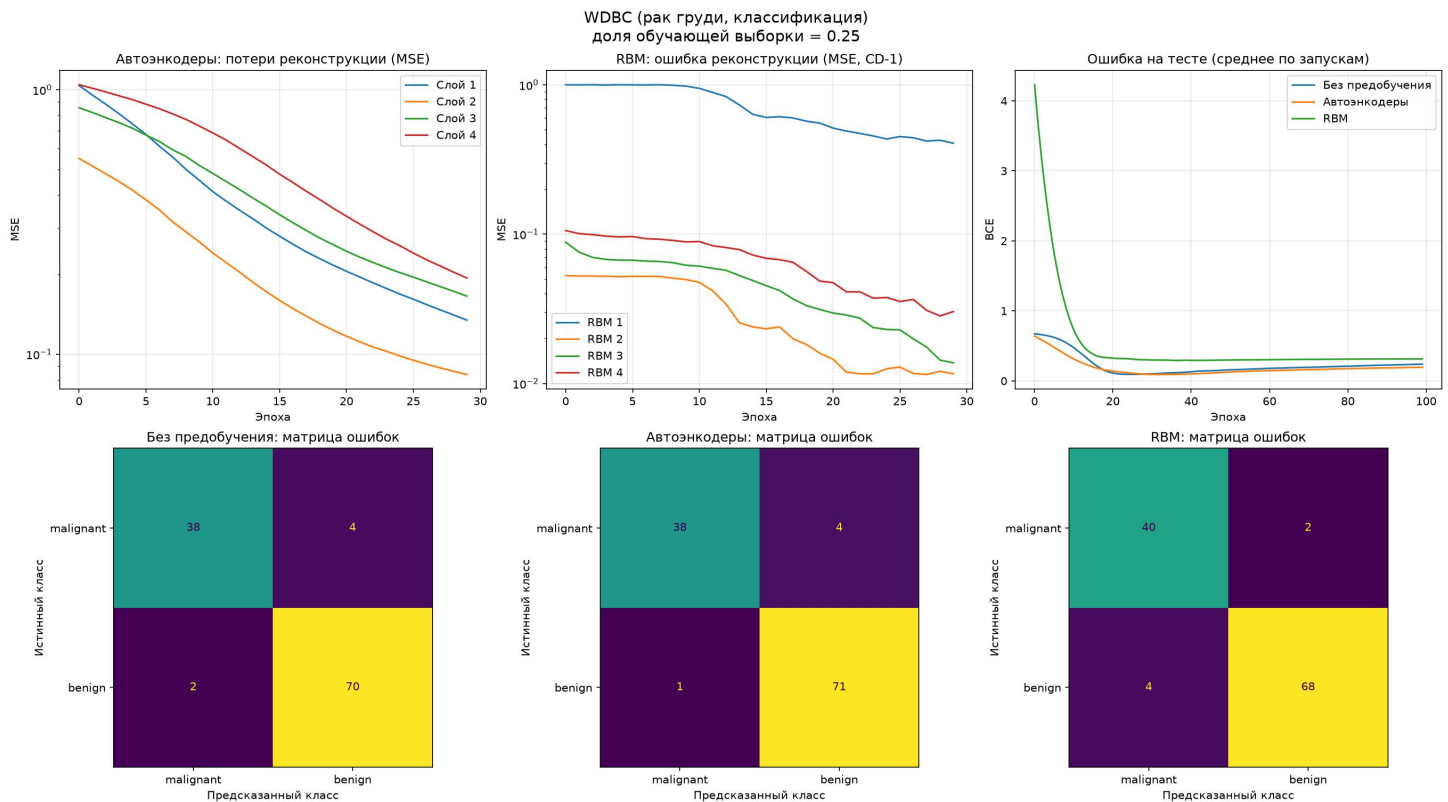
Таблица 1 – Результаты на CASP (тестовая выборка)

Доля train	Метод	MAE	RMSE	R ²	MAPE, %
100%	Без предобучения	2,994	4,118	0,546	63,9
100%	Автоэнкодеры + дообучение	3,107	4,174	0,534	68,0
100%	RBM + дообучение	3,190	4,251	0,516	73,0
5%	Без предобучения	3,845	4,841	0,373	91,5
5%	Автоэнкодеры + дообучение	3,988	4,940	0,347	97,2
5%	RBM + дообучение	4,203	5,121	0,298	103,2

Результаты сравнения:

- Лучшее качество показала сеть без предобучения. Автоэнкодерное предобучение чуть хуже (разница RMSE 0,057 на полной выборке, $p = 0,037$ по парному t-тесту). Предобучение RBM хуже заметнее: разница с базовой моделью 0,133 ($p = 0,0006$), с автоэнкодерами 0,076 ($p = 0,011$).
- На уменьшенной выборке (5%) ожидаемого выигрыша от предобучения тоже нет. Автоэнкодеры не отличаются от базовой модели значимо ($p = 0,083$), RBM значимо хуже ($p = 0,002$).
- Без дообучения с учителем (только предобучение и обучение выходного слоя) качество низкое: $R^2 = 0,20$ у автоэнкодеров и 0,05 у RBM. Оба метода не используют целевую переменную, поэтому дообучение обязательно.
- По графикам тестовой ошибки кривая RBM начинается выше остальных, но к концу обучения приближается к ним. Реконструкция RBM выходит на плато уже за 5–10 эпох, а автоэнкодеры продолжают снижать ошибку.





Сравнение результатов на выборке WDBC (данные из ЛР 2, классификация)

Схема эксперимента та же: архитектура 30-64-32-16-8-1, 5 разбиений со стратификацией, доли обучающей выборки 100% и 25%.

Таблица 2 – Результаты на WDBC (тестовая выборка)

Доля train	Метод	Accuracy	Precision	Recall	F1
100%	Без предобучения	0,9754	0,9727	0,9889	0,9807
100%	Автоэнкодеры + дообучение	0,9754	0,9727	0,9889	0,9807
100%	RBM + дообучение	0,9737	0,9752	0,9833	0,9792
25%	Без предобучения	0,9544	0,9645	0,9639	0,9638
25%	Автоэнкодеры + дообучение	0,9561	0,9646	0,9667	0,9652
25%	RBM + дообучение	0,8895	0,9037	0,9639	0,9253

Результаты сравнения:

- На полной выборке все три подхода дают практически одинаковое качество: F1 равен 0,981, 0,981 и 0,979, а различие в доли процента соответствует одному объекту. Статистически значимых различий нет ($p \geq 0,69$).
- На 25% выборки автоэнкодеры и базовая модель по-прежнему совпадают ($F1 = 0,965$ и $0,964$). RBM показывает меньшее значение F1 (0,925) и Accuracy (0,890), однако из-за малого размера выборки (≈ 113 объектов) и разброса между запусками различие незначимо ($p \approx 0,33$).
- Без дообучения качество заметно ниже (Accuracy 0,80 у автоэнкодеров и 0,90 у RBM на полной выборке), как и на CASP.
- WDBC хорошо разделима, поэтому все методы быстро достигают предела качества и предобучение не даёт преимуществ.

Выводы о проделанной работе: в работе реализовано послойное предобучение полносвязной сети [вход-64-32-16-8-1] с помощью стека RBM. Первый слой — гауссово-бинарная RBM, остальные — бинарные, обучение алгоритмом CD-1 по 30 эпох на слой. Результаты сравнены с обучением без предобучения и с автоэнкодерным предобучением (ЛР3) на CASP (регрессия RMSD) и WDBC (данные из ЛР2). Все методы стартовали с одинаковых начальных весов, эксперименты повторены на 5 разбиениях.

- **CASP, полная выборка.** Предобучение не дало выигрыша. R^2 равен 0,546 без предобучения, 0,534 с автоэнкодерами и 0,516 с RBM. RMSE равен 4,12, 4,17 и 4,25. RBM значимо хуже базовой модели ($p = 0,0006$) и автоэнкодеров ($p = 0,011$). Автоэнкодеры хуже базовой модели лишь на границе значимости ($p = 0,037$).
- **CASP, 5% выборки.** Ожидаемого преимущества на малых данных нет. R^2 равен 0,373, 0,347 и 0,298, RBM снова значимо хуже ($p = 0,002$).
- **WDBC, полная выборка.** Все три метода дают одинаковое качество: $F1 = 0,981, 0,981$ и $0,979$, различия в один объект и незначимы.
- **WDBC, 25% выборки.** $F1$ равен 0,964 (без предобучения), 0,965 (автоэнкодеры) и 0,925 (RBM). У RBM Ассигуру ниже на 6,5 п.п., но из-за большого разброса на 5 запусках различие незначимо ($p \approx 0,33$). Вероятная причина: на ~113 объектах RBM-слои не успели сойтись за 30 эпох.
- **Только предобучение, без дообучения.** Оно уступает дообучению с учителем у обоих подходов. Например, на CASP $R^2 = 0,20$ у автоэнкодеров и 0,05 у RBM. Оба метода не используют целевую переменную, поэтому дообучение обязательно.
- **Причины.** Сеть небольшая и мелкая, ReLU с Adam не страдает от затухающего градиента, а у CASP всего 9 признаков. Кроме того, RBM обучается на сигмоидных вероятностях, а затем веса переносятся в сеть с ReLU. Это несоответствие может частично объяснять худший результат RBM.

Для этих небольших табличных датасетов предобучение (автоэнкодерное и на RBM) не улучшило качество. Автоэнкодеры дали результат, близкий к базовой модели, RBM оказался хуже или равен ей. Такое предобучение оправданно для более глубоких сетей и сложных многомерных данных, например изображений.